SAVESTREET DOGS

COMPLETE MASTER IMPLEMENTATION PROMPT

Combined brief: layout repair • WhatsApp • PawPal chatbot • shop • adoption • admin • content • security • deployment

Instructions for Replit Agent: Read this complete prompt and implement all applicable requirements in the existing repository. Inspect first, preserve working functionality, test honestly, and do not claim unfinished work is complete.

SAVESTREET DOGS — COMPLETE MASTER IMPLEMENTATION PROMPT

Full Website Repair, Products, Shop, Adoption, Authentication, Chatbot, Content, UI/UX and Deployment

You are working on my existing SaveStreet Dogs website and repository:

GitHub: https://github.com/Nasty-Developer/Prince

Your task is to inspect, repair, improve, and complete the existing website from beginning to end.

I previously provided multiple requirements, and some were not implemented. This prompt combines ALL those requirements into one implementation task. Do not implement only the latest request or only update the homepage. Inspect the entire project and complete every applicable requirement below.

Most important rule: Do not rebuild the website from scratch. Work within the existing codebase, preserve working functionality, reuse the current architecture, and make changes carefully. Do not claim that a feature is completed unless you have implemented and tested it.

PHASE 1: AUDIT THE ENTIRE EXISTING PROJECT

Before changing code, inspect the full repository.

Identify:

- Frontend application and its entry point.
- All workspace packages and shared libraries.
- Existing pages, components, routes, and navigation.
- Current product model and Shop implementation.
- Puppy and adoption data models.
- Admin dashboard and authorization.
- Firebase Authentication, Firestore, and Storage integration.
- Existing cart and checkout functionality.
- Rescue reports, volunteer applications, and foster applications.
- Current image assets and upload functionality.
- Build scripts, environment variables, and deployment configuration.
- Existing tests and errors.

This project uses a pnpm workspace monorepo. Do not assume the root directory is the actual frontend application.

Inspect existing configuration and use the correct package scripts. Avoid creating duplicate applications, competing configurations, unnecessary dependencies, or parallel implementations of existing features.

Create a concise internal implementation checklist and work through all phases below.

PHASE 1A: URGENTLY REPAIR THE BROKEN WEBSITE LAYOUT AND PRODUCT IMAGE SIZING

The current website has a broken layout: product images can render excessively large, card alignment is inconsistent, and sections may overflow. A screenshot was provided zoomed out intentionally to show the overall problem. Do not interpret that browser zoom level as the intended website design. Test and tune the actual site at 100% browser zoom.

Root-cause investigation

Before editing, inspect the real application components, stylesheets, Tailwind classes, shared image components, and layout wrappers. Identify why product images are oversized instead of applying temporary CSS patches.

Check for:

- Missing or incorrect image width, height, max-width, or aspect-ratio constraints.
- Global `img` rules overriding component-specific styles.
- Incorrect grid/flex configuration or missing `min-width: 0`.
- Conflicting or duplicated CSS selectors.
- Incorrect image wrappers, component markup, or utility classes.
- Broken responsive breakpoints.
- Image dimensions or inherited styles from another section.
- CSS or component changes introduced by the current branch/merge.

Product image and card requirements

- Keep every product image inside its own image container and product card.
- Use consistent image-container dimensions and preserve source aspect ratios without distortion.
- Use `object-fit: contain` when the full product should remain visible; use `cover` only where cropping is appropriate.
- Apply `width: 100%`, `height: 100%`, `max-width: 100%`, and suitable `object-position` where appropriate.
- Prevent an image from determining or expanding the page/section width.
- Align product image, name, price, description, and action buttons consistently.
- Keep cards from overlapping or stretching beyond grid columns.
- Ensure different source image dimensions still produce consistent cards.
- Do not blur images unnecessarily or crop product details.

Example approach only; adapt selectors and values to the actual codebase rather than creating duplicate stylesheets:

```
.product-card {
min-width: 0;
overflow: hidden;
}

.product-image-container {
width: 100%;
height: 220px;
display: flex;
```

```
align-items: center;
justify-content: center;
overflow: hidden;
}

.product-image-container img {
display: block;
width: 100%;
height: 100%;
max-width: 100%;
object-fit: contain;
object-position: center;
}
```

Responsive layout requirements

- Desktop: show 3–4 evenly sized product cards per row where the available width supports it.
- Tablet: normally 2 cards per row.
- Mobile: 1–2 cards per row depending on available width and readability.
- Use the existing grid system or CSS Grid and consistent spacing.
- Use a centered content container with a suitable maximum width.
- Fix the hero text/image balance, puppy image sizing, navigation, buttons, forms, footer, and all affected sections.
- Remove accidental oversized margins, padding, widths, and heights.
- Prevent unintended horizontal scrolling, clipped text, overlapping controls, and broken mobile layouts.
- Preserve existing SaveStreet Dogs branding, colors, typography, content, and working features.

Required verification

Test at 100% browser zoom and representative mobile, tablet, laptop, and desktop widths. Confirm product images remain inside cards, grids align, no section causes horizontal overflow, and the browser console/build output has no errors caused by the changes. Fix the underlying implementation rather than using arbitrary offsets or excessive `!important` declarations.

PHASE 2: ADD THE REFLECTIVE COLLARS PRODUCT

I am attaching two photographs of the same product.

Both photographs represent ONE product. They must not become two separate products.

Product information

- Product name: Reflective Collars
- Price: ■60 per piece
- Category: Dog Accessories / Collars
- Currency: INR (■)
- Product images: Both attached photographs.

Use the actual attached images. Do not substitute unrelated stock photos or generate replacement product photographs.

A. Shop page

Add Reflective Collars to the existing Shop page.

Requirements:

1. Create one product record or listing.
2. Associate both attached photographs with the same product.
3. Display the product name, image, category, and ■60 per-piece price.
4. Provide a working product details page or existing equivalent.
5. Integrate the product with the current cart and checkout flow.
6. Allow customers to choose quantities.
7. Calculate totals correctly: ■60 × quantity.
8. Display prices in Indian rupees.
9. Preserve the existing cart, checkout, and payment functionality.
10. Use the current database and product schema wherever possible.
11. Make the product manageable through the existing admin panel if product management is implemented.
12. Ensure the product and its image URLs persist after refreshing the website or redeploying it.

Do not create duplicate product records, duplicate listings, or fake inventory values.

If the application uses a database, implement the product using its existing persistence mechanism. If the product must be seeded, make the operation idempotent so it does not create duplicates.

B. Homepage product section

Immediately below the homepage hero section, add a polished product section titled:

"Shop for Dogs"

Alternatively, use **"Support Street Dogs"** if that better matches the existing design.

Display Reflective Collars with:

- Product photograph.
- Product name.
- Price: ■60 / piece.
- A working "View Product" button.
- A working "Shop Now" link to the full Shop page.

The homepage and Shop page must use the same product data. Do not hardcode two separate copies.

Keep the existing hero section and other working sections intact.

C. Product presentation

- Use both attached photographs appropriately in the product card and product details.
- Optimize images for loading without making them blurry.
- Preserve their aspect ratios.
- Prevent image stretching and layout shifts.
- Ensure the product looks good on mobile, tablet, and desktop.
- Keep the design consistent with the SaveStreet Dogs brand.

For this task, Reflective Collars is the confirmed product and ■60 is the confirmed price. Do not invent other products or prices. Preserve other legitimate existing products already present in the database.

PHASE 3: REPAIR ALL WEBSITE PAGES AND ROUTES

Some website pages are not loading correctly. The Stories page previously returned a 404 Not Found error.

Inspect all routes, pages, navigation links, buttons, and internal links.

Fix:

- Stories page.
- Learn page.
- Shop and product details.
- Puppy listing and adoption pages.
- Contact page.
- Sign-in and sign-up pages.
- Admin dashboard routes.
- Volunteer and foster pages.
- Rescue reports.
- Any other existing page that is broken.

Requirements:

1. Every visible navigation item must lead to a valid destination.
2. Do not hide broken pages to make the site appear functional.
3. Fix direct navigation to nested URLs.
4. Fix refresh behavior on nested routes.
5. Ensure browser back and forward navigation works.
6. Show appropriate not-found pages for genuinely unknown routes.
7. Avoid blank screens, broken imports, and runtime errors.
8. Test desktop and mobile navigation.

Inspect the Vercel configuration and ensure the frontend's actual build output is served correctly.

The existing root configuration must not accidentally publish an empty directory or the wrong workspace package. Verify the correct package, build command, output directory, and SPA rewrite for the actual application.

PHASE 4: CREATE THE STORIES SECTION WITH NINE STORIES

Implement a complete Stories page featuring **nine meaningful, original stories related to street dogs, animal rescue, adoption, fostering, and community support**.

Each story should include:

- Title.
- Summary.
- Relevant image.
- Readable article content.
- Working detail view or article-reading experience.
- Appropriate links to related website sections.

Use actual verified rescue stories from the existing project when available.

If actual rescue records are unavailable, create clearly labelled illustrative or educational stories rather than inventing rescue incidents, real people, adoption outcomes, or fundraising results and presenting them as facts.

The Stories page must:

- Have a polished, responsive card layout.
- Support working detail navigation.
- Display images correctly.
- Have clear calls to action.
- Work when opened directly or refreshed.
- Link to relevant adoption, volunteer, foster, and rescue pages.

Do not use nine copies of the same story or generic placeholder cards.

PHASE 5: IMPLEMENT THE LEARN SECTION

Create a useful educational section with nine original articles.

1. How to Help a Stray Dog Safely.
2. What to Do When You Find an Injured Street Dog.
3. Responsible Street Dog Feeding.
4. Understanding Street Dog Body Language.
5. Why Vaccination and Sterilization Matter.
6. How to Adopt a Rescue Dog.
7. Preparing Your Home for an Adopted Dog.
8. How to Become a Dog Foster Volunteer.
9. Helping Puppies Stay Safe on the Streets.

Every article must have:

- A title and summary.
- Original, readable content.
- Relevant imagery.
- A working detail page or reading experience.
- Appropriate internal links.
- A responsive layout and accessible typography.

Do not copy existing articles from other websites.

Keep animal welfare advice responsible. Do not recommend dangerous procedures or unverified medical treatments. Direct urgent medical situations to qualified veterinary professionals.

The Learn section must be accessible through the website's existing navigation and must not return 404 errors.

PHASE 6: ADD WHATSAPP INTEGRATION USING THE CONFIRMED NUMBER

The confirmed WhatsApp contact number is **8955438209**.

Use this international-format WhatsApp URL everywhere:

`https://wa.me/918955438209`

Requirements:

- Update the existing floating WhatsApp button; do not create a duplicate if one already exists.
- Update every relevant public-facing WhatsApp contact action to use this same number.
- Centralize the number in one shared configuration or contact utility so it can be changed in one place later.
- Use properly URL-encoded prefilled messages, for example:

`https://wa.me/918955438209?text=Hello%2C%20SaveStreet%20Dogs%21`

- General contact messages should be friendly and relevant.
- Puppy adoption messages should include the puppy's name when available.
- Rescue-report messages should include a suitable rescue enquiry template.
- Product enquiries should include the product name when applicable.
- Do not invent or retain an old WhatsApp number in any active contact link.
- Do not change unrelated phone numbers or email addresses.
- The floating button must have a recognizable icon, accessible label, keyboard focus feedback, and responsive sizing.
- Keep it fixed near the bottom-right corner while avoiding forms, important buttons, mobile navigation, and the PawPal assistant.
- Use an appropriate stacking order so the WhatsApp button and chatbot never overlap.
- Test the generated links on desktop and mobile. Do not claim a link was externally tested unless it actually was.

PHASE 7: IMPLEMENT PAWPAL, A PERSONAL CHAT ASSISTANT THAT WORKS WITHOUT API KEYS

I do not currently have API keys for OpenAI, Gemini, or another AI provider.

Give the assistant the friendly identity **PawPal — the SaveStreet Dogs Assistant**. Build a premium, fully interactive chatbot UI, not merely a static mockup. It must work without paid API credentials.

PawPal visual and interaction requirements

- Add a floating launcher and polished chat panel/window.
- Use the existing SaveStreet Dogs branding: warm, welcoming animal-welfare styling, soft neutral backgrounds, restrained green accents, elegant typography, rounded corners, and subtle shadows.
- Use a friendly avatar from an existing project asset or a simple icon/CSS illustration.
- Include a header with PawPal's name, short description, and close button.
- Clearly distinguish user and assistant message bubbles.
- Include a welcome message, suggested prompt chips, message input, send button, and a conversation reset/clear control.
- Add a lightweight typing indicator while a local response is prepared and timestamps if they fit the current design.
- Support Enter to send and Shift+Enter for a newline where a multiline input is used.
- Provide clear empty, fallback, loading, and error states.
- Use smooth, lightweight open/close and message transitions; respect reduced-motion preferences.
- Make the panel excellent on mobile, tablet, laptop, and desktop.
- Ensure the launcher and panel do not overlap the WhatsApp button, important page controls, forms, or mobile navigation.

- Use accessible labels, keyboard navigation, visible focus states, readable contrast, and appropriate touch targets.

PawPal response and navigation requirements

PawPal should provide local responses for greetings, thanks, general website questions, SaveStreet Dogs' purpose, puppy adoption, rescue reports, volunteering, fostering, products, donations, stories, Learn articles, contact information, and the Reflective Collars product priced at ■60 per piece.

Use actual existing site content and data. Never invent puppy names, availability, ages, locations, product inventory, rescue outcomes, payment confirmations, or other factual claims. When a question cannot be answered, use a friendly, honest fallback.

Where the relevant feature exists, offer working actions/links to explore puppies, submit adoption requests, report a dog, browse the shop, view a product, apply to volunteer, apply to foster, read Stories/Learn content, and contact SaveStreet Dogs through the configured WhatsApp link.

The chatbot must not falsely claim to be a full generative AI model. Explain through suitable UI copy that it is a local website assistant when relevant.

Chat interface

Include:

- Floating chat launcher.
- Modern chat window.
- Welcome message.
- Suggested questions.
- Open and close controls.
- Clear messages and responses.
- Loading feedback when useful.
- Accessible keyboard and touch interactions.
- Responsive mobile and desktop layout.

Knowledge base

Use actual website content to answer common questions about:

- SaveStreet Dogs.
- Available puppies and adoption.
- Adoption application process.
- Rescue reports.
- Volunteering.
- Fostering.
- Donations and ways to help.
- Shop products.
- Reflective Collars and the ■60 price.
- Stories and Learn articles.
- Contact information.

Use reliable local keyword matching and intent-based responses.

Provide links to relevant website pages and a sensible fallback when the chatbot cannot answer a question.

Do not fabricate puppy availability, rescue cases, payment confirmations, medical advice, or other factual claims.

Technical requirement: Without an AI model or external AI service, this is a local knowledge-based chatbot, not a generative AI model. Do not falsely advertise it as a live AI service.

Structure the code so a real AI provider can be integrated later through a secure backend if I obtain an API key.

Do not require an API key for the current implementation.

Ensure the chatbot and WhatsApp button do not overlap at any supported screen size.

PHASE 8: FIX FIREBASE SIGN-IN AND SIGN-OUT

The website already has Sign In and Sign Out options. Inspect and repair the existing authentication implementation.

Authentication requirements

- Reuse the existing Firebase project.
- Reuse the existing Firebase configuration and environment variables.
- Do not replace valid values with dummy values.
- Do not create a new Firebase project unnecessarily.
- Fix sign-in, sign-out, authentication state persistence, loading states, and error handling.
- Ensure navigation displays the correct authentication state.
- Implement appropriate redirects.
- Handle expired sessions and failed authentication gracefully.

Admin security

- Protect admin routes.
- Verify administrator permissions through trusted authorization checks.
- Review Firestore security rules and Storage security rules.
- Prevent ordinary users from accessing administrative functionality.
- Prevent unauthorized changes to products, puppy records, orders, and website content.
- Restrict file uploads to authorized users and appropriate file types and sizes.

Do not rely only on hiding admin buttons in the frontend.

Firebase web configuration values may be used in client applications, but private service-account credentials and server secrets must never be exposed in frontend code.

If an existing configuration value is missing or invalid, report exactly which setting needs attention. Do not invent credentials or silently replace the existing Firebase project.

PHASE 9: REPAIR AND PRESERVE THE ADMIN PANEL

Inspect the existing administrator dashboard and make sure its current features work.

Preserve and verify relevant existing modules, including:

- Dashboard.
- Puppy management.
- Adoption requests.
- Product management.
- Orders.
- Rescue reports.
- Volunteer applications.
- Foster applications.
- Stories.
- Website content.
- Settings.

Do not assume that every module is already implemented. Inspect the actual code and repair or complete existing functionality where applicable.

For product management, verify that the Reflective Collars product can be managed through the existing system if the module supports product management.

Ensure admin changes persist and appear on the public website where appropriate.

Do not create insecure shortcuts, bypass authentication, or expose sensitive administrative data.

PHASE 10: ADD CONFIRMED SOCIAL MEDIA LINKS

Use the actual SaveStreet Dogs social media URLs below.

Instagram

https://www.instagram.com/savestreetdogs18/

Facebook

https://www.facebook.com/profile.php?id=61591879406906

These links are confirmed. Replace the previous temporary social media placeholders with these actual URLs.

Requirements:

1. Add Instagram and Facebook links to the website footer.
2. Use recognizable Instagram and Facebook icons.
3. Add the links to the contact section if appropriate.
4. Open external profiles in a new tab.
5. Use safe link attributes, including `rel="noopener noreferrer"` when using `target="_blank"`.
6. Ensure icons have accessible labels.
7. Keep the icons responsive and visually consistent with the website.
8. Store the URLs in a central configuration or existing website settings.
9. Remove obsolete placeholders throughout relevant components.
10. Test both links on desktop and mobile.

Do not invent other usernames or account URLs.

Do not build an Instagram feed that requires unavailable API credentials. Simple working links to the confirmed profiles are sufficient.

PHASE 11: VERIFY THE PUBLIC CONTACT EMAIL

The intended public contact email for SaveStreet Dogs is:

vexaxam466@caps7.com

Inspect the existing public-facing contact information and replace obsolete public contact email addresses with this address where appropriate.

Requirements:

- Update the contact section and public contact links where applicable.
- Use a working `mailto:` link.
- Keep the address consistent across relevant public-facing components.
- Do not replace Firebase login emails, administrator credentials, or private configuration values with the public contact email.
- Do not invent a phone number or other contact details.

PHASE 12: IMPROVE THE ENTIRE WEBSITE DESIGN

The website currently feels too static. Improve its visual quality while preserving the SaveStreet Dogs identity.

Aim for a clean, modern, polished, professional animal-welfare website.

Improve:

- Typography and visual hierarchy.
- Section spacing.
- Navigation and footer.
- Hero section and calls to action.
- Puppy and adoption cards.
- Product cards and shopping interactions.
- Stories and educational article layouts.
- Form design.
- Loading, empty, and error states.
- Consistent buttons, icons, and interaction feedback.

Animations

Add subtle, purposeful animations:

- Scroll-triggered reveal effects.
- Smooth hover effects on cards and buttons.
- Smooth transitions for menus and dialogs.
- Appropriate page transitions where compatible with the current routing.
- Chatbot opening and closing transitions.

- Subtle interaction feedback.

Avoid excessive motion, distracting effects, unnecessary libraries, and performance regressions.

Respect the user's reduced-motion preference.

Do not redesign everything arbitrarily. Preserve existing branding and working components unless a change is necessary for consistency or usability.

PHASE 13: MAKE THE ENTIRE WEBSITE RESPONSIVE

Test the website at mobile, tablet, laptop, and desktop widths.

Pay particular attention to:

- Navigation and mobile menus.
- Hero section.
- Homepage product section.
- Shop and product details.
- Puppy listings and adoption pages.
- Stories and Learn.
- Contact and volunteer forms.
- Sign-in and sign-up.
- Admin dashboard tables and forms.
- WhatsApp and chatbot controls.
- Footer and social media links.

Requirements:

- No unintended horizontal scrolling.
- No clipped text.
- No distorted product or puppy images.
- No overlapping floating controls.
- No inaccessible buttons or links.
- No unusable forms on small screens.
- Proper image sizing and aspect ratios.
- Comfortable typography and touch targets.
- Functional navigation at supported screen sizes.

Do not consider responsiveness complete merely because the homepage fits on a mobile screen. Check the actual pages and their interactions.

PHASE 14: PRESERVE ALL EXISTING DATA AND FEATURES

Do not break existing:

- Puppy listings.
- Adoption requests.

- Admin authentication and permissions.
- Product management.
- Shop and cart.
- Checkout and payment flows.
- Rescue reports.
- Volunteer applications.
- Foster applications.
- Firebase integration.
- Existing database records.
- Image upload functionality.
- Deployment configuration.

Reuse existing routes, data models, shared components, workspace packages, and services wherever possible.

Do not delete existing records or reset databases during implementation.

Avoid unnecessary schema migrations. If a migration is required, make it backward-compatible and explain it.

Do not add fake puppies, fake testimonials, fake adoption statistics, fabricated rescue claims, or invented product inventory.

Preserve all legitimate existing products and their data. Add the confirmed Reflective Collars product without replacing the entire product catalogue.

PHASE 15: BUILD, TEST, AND DEPLOY CORRECTLY

The repository is a pnpm workspace monorepo. Determine the correct application package before running commands.

Inspect:

- `package.json`.
- `pnpm-workspace.yaml`.
- `pnpm-lock.yaml`.
- Existing Vercel configuration.
- Existing Render or backend configuration.
- Build scripts and output directories.
- Required environment variables.
- Existing tests.

A previous deployment configuration used the following frontend build command and output directory:

Build command:

```
pnpm --filter @workspace/savestreet-dogs-main run build
```

Output directory:

```
artifacts/savestreet-dogs-main/dist/public
```

These are historical settings, not permission to assume the current repository still uses them. Verify the actual package name and build output before applying them.

If the correct Vercel configuration needs an SPA fallback, configure it without interfering with existing API routes or backend services.

Required testing

1. Install dependencies using the project's existing package manager and lockfile.
2. Run the appropriate type-check and build commands.
3. Fix errors introduced by the implementation.
4. Test the homepage and hero section.
5. Verify Reflective Collars appears below the hero.
6. Verify Reflective Collars appears on the Shop page as one product with two images.
7. Test product details, quantity selection, price calculation, and existing cart integration.
8. Test puppy listings and adoption requests.
9. Test Stories and all nine story pages or detail views.
10. Test Learn and all nine educational articles.
11. Test Sign In and Sign Out.
12. Verify admin authorization and existing security rules.
13. Test the chatbot without any AI API key.
14. Verify WhatsApp configuration and link generation.
15. Test both confirmed social media URLs.
16. Verify the public contact email.
17. Test mobile menus and all floating controls.
18. Check browser console errors, missing images, and broken links.
19. Confirm that persistent data and image URLs work after refresh.
20. Confirm the frontend build output and deployment configuration are correct.

Do not claim to have tested live Firebase authentication, production deployment, or external links unless those checks were actually performed.

If a test requires unavailable credentials or access, state the exact limitation and what remains to be verified.

PHASE 16: CHANGE MANAGEMENT AND SAFE GIT WORKFLOW

- Inspect `git status`, the current branch, recent commits, and existing diffs before editing.
- Preserve user changes, existing data, and all recovery branches.
- Do not run destructive commands such as `git reset --hard`, `git clean -fd`, force-push, or `git rebase --skip` as shortcuts.
- Do not blindly choose one side of merge conflicts in application code, API schemas, package manifests, or lockfiles. Understand the changes and resolve conflicts deliberately.
- Do not discard local changes just to make a build pass.
- Do not commit secrets, private service-account credentials, `.env` files, or generated sensitive data.
- Do not automatically commit or push unless explicitly instructed in the current task. If asked to commit/push, stage only intended files, review the diff, report the commit hash and push result, and never claim success without confirmation.
- If the repository is in an unfinished merge/rebase state, inspect and report it before attempting further Git operations.
- Keep a clear backup/recovery path before any risky repository operation.

FINAL IMPLEMENTATION PRIORITIES

Complete the work in this order while keeping the entire task in scope:

Priority 1: Audit the codebase, inspect repository/merge state safely, fix build problems, broken routes, Firebase authentication, and the oversized-image/responsive-layout bug.

Priority 2: Implement Reflective Collars with both attached images as one product, persistent product data, homepage placement immediately below the hero, and Shop integration.

Priority 3: Repair the admin panel and preserve the adoption, database, and existing product systems.

Priority 4: Complete Stories with nine meaningful stories and Learn with nine original educational articles.

Priority 5: Implement the WhatsApp floating button using 8955438209 (international URL `https://wa.me/918955438209`) and the polished PawPal local knowledge-based chatbot.

Priority 6: Replace social media placeholders with the confirmed Instagram and Facebook links. Verify public contact information.

Priority 7: Improve the design, animations, accessibility, and responsiveness across the entire website.

Priority 8: Test the build, routes, forms, product flows, authentication, and deployment configuration.

FINAL REPORT REQUIRED

When finished, provide a clear implementation report containing:

- Features completed.
- Bugs fixed.
- Product and image integration details.
- Pages and routes tested.
- Firebase and admin security checks performed.
- Responsive testing performed.
- Build and deployment status.
- Remaining errors or blocked tests.
- Any configuration values still required from me.

Do not mark unfinished work as completed.

FINAL GOAL

Deliver a fully improved SaveStreet Dogs website that preserves existing functionality and provides:

- A reliable animal adoption and rescue experience.
- A working Shop with Reflective Collars priced at ■60 per piece, using both attached images for one product.
- A homepage product section directly below the hero.
- Working stories and educational articles.
- Functional Firebase authentication and a secure admin panel.
- A WhatsApp contact button.
- A local chatbot that requires no AI API key.

- The confirmed Instagram and Facebook links.
- Consistent public contact information.
- Smooth, purposeful animations.
- Responsive layouts across all pages.
- A verified, correctly configured build and deployment.

Inspect first. Implement carefully. Test everything. Preserve existing working features. Do not stop after changing only the homepage or only the social media links. Complete every applicable requirement in this master prompt.